IEICE TRANS. COMMUN., VOL.E109–B, NO.2 FEBRUARY 2026

PAPER

Unsupervised Real-Time In-Kernel Intrusion Detection System Using Autoencoders and eBPF*

Hotaka TAGUCHI[†a)], **Takanori HARA**[†b)], ***Members***, ***and*** **Shoji KASAHARA**[†c)], ***Fellow***

SUMMARY Traditional intrusion detection systems (IDSs), leveraging machine learning (ML) algorithms, have improved the detection accuracy of unknown attacks by continuously updating ML models but have underestimated the context switching overhead between kernel and user spaces. To address this issue, existing studies have implemented real-time IDSs using neural networks (NNs) in the kernel space by offloading the quantized models trained with post-training quantization (PTQ) to extended Berkeley Packet Filter (eBPF). However, they cannot fine-tune the model parameters through the additional training because the PTQ applies the quantization to the trained model. In addition, their IDSs are based on supervised learning, which requires a large amount of labeled data. In this paper, we propose a real-time in-kernel IDS leveraging eBPF, unsupervised learning, and quantization-aware training (QAT) to enhance continuous learning. Evaluation results demonstrate that the proposed in-kernel IDS exhibits almost the same detection accuracy as the traditional user-space IDS. From the viewpoint of the packet processing speed, the proposed in-kernel IDS can serve 224 K packets per second while the user-space IDS can only serve 3.2 K packets per second.

key words: *intrusion detection system (IDS), extended Berkeley packet filter (eBPF), quantization-aware training (QAT), unsupervised learning*

1. Introduction

Traditional *intrusion detection systems* (IDSs) are based on signature-based detection, which compares network traffic with predefined rules and patterns stored in databases [2]. The signature-based IDSs cannot support unknown attacks whose signatures have not been sufficiently analyzed. They degrade processing performance due to the huge number of rules. To address these issues, anomaly-based IDSs, leveraging *machine learning* (ML), have attracted attention over the years [3], [4]. The anomaly-based IDSs can overcome these limitations by continuously updating the detection model with the latest attack patterns. Many researchers have addressed anomaly-based IDSs using ML algorithms, such as *multi-layer perceptrons* (MLPs) and *autoencoders* (AEs). However, most of them ignore the cost of context switching between user and kernel spaces [5].

In order to mitigate the context switching cost, in-kernel IDSs have been proposed [6]–[10], which offload functionalities of IDSs to *extended Berkeley Packet Filter* (eBPF). eBPF, which is an abstract virtual machine that executes user-specified programs in the kernel space, processes packets in the kernel space by the verified program and eliminates overheads caused in a packet processing path (i.e., *sk_buff*, a *protocol stack*, *traffic control* (TC), and a *socket*) [11], [12]. However, eBPF is imposed with severe constraints, such as the prohibition of floating-point arithmetic and the limitations of instruction counts and stack space, to avoid kernel crashes. Therefore, eBPF programs must be verified by the eBPF verifier before they are loaded into tracepoints.

Several studies have implemented MLPs in eBPF programs for anomaly-based IDSs [8]–[10]. In [8], the authors have employed fixed-point arithmetic to implement MLPs in the eBPF program instead of floating-point arithmetic to avoid the eBPF constraints. However, this approach cannot handle a large-scale MLP model using *neural networks* (NNs) due to the limitations of stack spaces. The research [9], [10] can overcome this issue by applying *post-training quantization* (PTQ) to the trained NN model. However, this approach has the following limitations: (1) Supervised learning needs to prepare the training data with labels in advance, which requires huge annotation costs. (2) PTQ applies quantization to the trained model, which prevents fine-tuning the model parameters in an online manner. On the contrary, *quantization-aware training* (QAT), one of the representative quantization methods, can adjust the quantization parameters during the training process [13], [14], which enables the model parameters to be continuously updated.

To address these issues, we propose a real-time in-kernel IDS leveraging eBPF, AEs, and QAT to enhance continuous training. The proposed method consists of three parts: data collection, training in the user space, and anomaly detection in the kernel space. In the data collection phase, the eBPF program inspects the benign packet from trusted clients in the white list and the user-space application extracts its features from the packet flow. In the training phase, the AE model is trained with QAT in the user space, given the features as inputs. The trained parameters are quantized and then shared with the eBPF program running on the kernel space via *BPF maps*, which enhance data exchange between the user space and the kernel space. In the anomaly detection phase, the eBPF program calculates the reconstruction error between the input and the output of the AE model in the kernel space. If the reconstruction error is larger than the predefined threshold, the packet is regarded as an anomaly,

Manuscript received March 3, 2025.
Manuscript revised June 11, 2025.
Manuscript publicized August 19, 2025.

[†]Division of Information Science, Nara Institute of Science and Technology, Ikoma-shi, 630-0192 Japan.

*This paper is an extended version of the paper that was presented at 2024 International Conference on Emerging Technologies for Communications (ICETC2024) [1].

a) E-mail: taguchi.hotaka.tc3@is.naist.jp
b) E-mail: hara@ieee.org
c) E-mail: kasahara@ieee.org

DOI: 10.23919/transcom.2025EBP3028

Copyright © 2026 The Institute of Electronics, Information and Communication Engineers

Table 1 A comparison with in-kernel IDSs.

Ref.	IDS	ML model	Packet processing mechanism	Quantization
[15]	Signature-based	-	eBPF/XDP	-
[16]	Signature-based	-	eBPF/XDP	-
[17]	Anomaly-based	-	eBPF	-
[18]	Anomaly-based	K-means (unsupervised)	eBPF	-
[6]	Anomaly-based	DT (supervised)	eBPF	-
[7]	Anomaly-based	SVM and RF (supervised)	eBPF	-
[8]	Anomaly-based	MLP (supervised)	eBPF	-
[9]	Anomaly-based	MLP (supervised)	eBPF/XDP/AF_XDP	PTQ
[10]	Anomaly-based	MLP (supervised)	eBPF/XDP	PTQ
Proposed method	Anomaly-based	AE (unsupervised)	eBPF	QAT

otherwise, it is benign.

The main contributions of this paper are as follows:

1. To the best of our knowledge, this is the first work to implement an unsupervised in-kernel IDS using AEs and QAT. This approach encompasses life cycles of data collection in the user space, training in the user space, and inference in the kernel space, enhancing continuous learning.
2. Through the evaluation, the proposed in-kernel IDS achieves almost the same detection accuracy as the existing user-space IDS. From the viewpoint of the packet processing speed, the proposed in-kernel IDS exhibits much higher throughput than the existing user-space IDS. The in-kernel IDS can serve 224 K packets per second at a single CPU core while the user-space IDS can only serve 3.2 K packets per second.
3. The proposed in-kernel IDS can continuously learn the model parameters and the quantization parameters without annotations thanks to both AEs and QAT. The evaluation result demonstrates that the model accuracy can be improved by fine-tuning the pretrained model parameters with the different traffic patterns.

In the conference version of this paper [1], we have presented the performance difference among the floating-point model, the quantized model using PTQ, and the quantized model using QAT in terms of the detection accuracy and the computation time by primarily focusing on the training phase. In this paper, we have extended the conference version by adding the following points: (1) We implement the in-kernel AE model trained with QAT for the anomaly-based IDS in the eBPF program in addition to the data collection mechanism. (2) The evaluation experiments have been conducted in terms of not only the detection accuracy but also the packet processing performance.

The rest of this paper is organized as follows. Section 2 gives the related work. Section 3 provides the preliminaries used in this paper. Section 4 introduces the proposed in-kernel IDS using QAT and AEs. In Sect. 5, we demonstrate the evaluation results in terms of detection performance and packet processing ability. Finally, Sect. 6 gives the conclusion.

2. Related Work

There have been many studies on traditional IDSs running on the user space [3], [4]. However, most of them ignore the context switching cost between the user space and the kernel space [5]. To address this issue, in-kernel IDSs have been proposed [6], [8]–[10], [15]–[19], which offload the functionalities of the IDS to the kernel space using the eBPF. Table 1 summarizes the related work on in-kernel IDSs. In [15], the authors analyzed signatures of *distributed denial of service* (DDoS) attacks and proposed an eBPF and/or *express data path* (XDP) empowered signature-based IDS using the rules based on the analyzed results. Choe et al. proposed an eBPF/XDP-based signature-based IDS that filters specific IP addresses [16].

Several studies offload ML algorithms to anomaly-based in-kernel IDSs using eBPF and/or XDP. These studies assumed that the training is executed in the user space while the inference is executed in the kernel space to eliminate the context switching cost. Hadi et al. proposed a rule-based in-kernel IDS, which classifies incoming packets into benign or malicious packets based on the rules defined in the user space [17]. The rules are updated by the ML algorithm running on the user space. Demoulin et al. proposed an in-kernel IDS using unsupervised learning (i.e., a k-means algorithm) [18]. Bachl et al. proposed a flow-based in-kernel IDS using a decision tree (DT) algorithm [6]. Anand et al. further applied *support vector machine* (SVM) and *random forest* (RF) algorithms to in-kernel IDSs assisted by eBPF [7].

The following studies implemented NNs in the eBPF program to improve the detection accuracy of the in-kernel IDSs. Gallego-Madrid et al. proposed an in-kernel IDS using fixed-point MLPs under IoT environments [8]. In order to maintain high inference accuracy, fixed-point arithmetic requires a sufficient large bitwidth to represent the weights and activations of the NN model. However, the eBPF constraints refuse to adopt the large bitwidth, which limits the scale of the NN model. Hara and Sasabe proposed a lightweight in-kernel IDS by applying a model compression technique (i.e., PTQ) to the NN model trained in the user space [9], [19]. In addition, they investigated the performance of in-kernel IDS using eBPF, XDP, and AF_XDP [9]. This approach can overcome the limitation of the scale of the NNs. However, it cannot handle dynamic model updates in a thread-safe

manner. In [10], Zhang et al. addressed this issue by implementing a thread-safe model updating mechanism without an explicit spin lock.

These studies cannot dynamically fine-tune the trained model parameters under scenarios that require continuous learning because they apply PTQ to the trained model. Furthermore, supervised learning requires annotation processes to prepare a large amount of labeled data in advance. In this paper, a real-time in-kernel IDS is proposed using the combination of unsupervised learning (i.e., AEs) and QAT. The proposed scheme enables continuous learning of the IDS without annotations.

3. Preliminaries

In this section, we elaborate on the elemental techniques to realize the proposed in-kernel IDS. Table 2 gives the notations used in this paper.

3.1 Packet Processing Mechanisms

Packet processing in Linux consists of four main stages: *sk_buff*, TC, a *protocol stack*, and a *socket*. In traditional user-space packet processing, upon the arrival of a new packet at the network interface card (NIC), the packet bytes are first copied to the kernel memory as a *sk_buff* structure. Next, the packet is passed to the TC, which performs packet classification and filtering. The packet is further passed to the protocol stack for validation of the protocol headers (e.g., IP header, TCP/UDP header, and so on). Within the socket, the packet bytes residing in the kernel memory are copied to the user-space memory via a system call because the application running in the user space cannot directly access the kernel memory. These stages cause the overhead of context switching, which is one of the factors for latency.

Table 2 Notations.

Symbol	Description
L	Number of layers
D	Number of features
$\boldsymbol{x}$	D-dimensional feature vector
$\boldsymbol{s}^{\min}$, $\boldsymbol{s}^{\max}$	D-dimensional normalization parameters
N	Batch size
$M^{(l)}$	Dimension of the lth layer
$\sigma(\cdot)$	Activation function
$\mathbf{X}^{(l)}$	Input matrix of the lth layer, $\mathbf{X}^{(l)} = (x_{i,j}) \in \mathbb{R}^{N \times M^{(l)}}$
$\mathbf{W}^{(l)}$	Weight matrix of the lth layer, $\mathbf{W}^{(l)} = (w_{i,j}) \in \mathbb{R}^{M^{(l)} \times M^{(l+1)}}$
$\mathbf{Y}^{(l)}$	Output matrix of the lth layer, $\mathbf{Y}^{(l)} = (y_{i,j}) \in \mathbb{R}^{N \times M^{(l+1)}}$
$\mathbf{X}_q^{(l)}$	Quantized input matrix of the lth layer
$\mathbf{W}_q^{(l)}$	Quantized weight matrix of the lth layer
$\mathbf{Y}_q^{(l)}$	Quantized output matrix of the lth layer
$s_x^{(l)}$, $s_w^{(l)}$	Scaling factors of the input and weight matrices with the lth layer
$z_x^{(l)}$, $z_w^{(l)}$	Zero points of the input and weight matrices with the lth layer
e	Reconstruction error
θ	Threshold of the reconstruction error

In contrast to traditional packet processing in the user space, eBPF programs offloaded to the TC can process packets before they reach the protocol stack. This mechanism allows eBPF programs to eliminate the overhead between the kernel space and the user space. However, eBPF programs have the following strict constraints: (1) The instruction counts and the jump counts are restricted to 1 M and 8192, respectively. (2) The stack space is limited to 512 bytes. (3) Floating-point arithmetic, unbounded loops, non-static global variables, and variadic functions are prohibited. (4) Array bound checks are enforced.

These restrictions make it difficult to implement ML algorithms in the eBPF programs. A BPF map is a key-value store, allowing the data to be shared between the user space and the kernel space. This mechanism contributes to saving stack space and gives alternatives to non-static global variables. In this paper, we use BPF maps to share the parameters of the trained model between the user space and the kernel space. In Sects. 3.2 and 3.3, we elaborate on how these restrictions are addressed to implement ML-based IDSs in eBPF programs.

3.2 Fixed-Point Arithmetic

In general, ML algorithms use floating-point arithmetic, which is not supported in eBPF due to the kernel stability and safety. Therefore, the fixed-point arithmetic is used in the eBPF program instead of the floating-point arithmetic. The fixed-point number can be represented by an integer value in which the lower n bits are the decimal part. A floating-point value x_{float} can be converted to a fixed-point value x_{fixed} with the lower n bits of decimal part as follows:

$$x_{\text{fixed}} = \text{round}(x_{\text{float}} \times 2^n), \tag{1}$$

where $\text{round}(x)$ is a function that rounds x to the nearest integer.

The fixed-point arithmetic obeys the following arithmetic rules [20], [21]: (1) The sum of two fixed-point numbers is a fixed-point number. (2) The product of a fixed-point number and an integer gives a fixed-point number. (3) Multiplying two fixed-point numbers with an n-bit shift right operation provides a fixed-point number.

3.3 Model Quantization

3.3.1 Quantization

Quantization is a process of mapping a real-valued number to a lower-bitwidth number, resulting in the reduction of CPU and memory consumption [22], [23]. Specifically, quantization maps a real-valued number $x_r \in [\beta, \alpha]$ to a b-bit unsigned integer $x_q \in \{0, 1, \ldots, 2^b - 1\}$ as follows:

$$\begin{aligned} x_q &= \text{quantize}(x_r, s_x, z_x) \\ &= \text{clip}\left(\text{round}\left(\frac{x_r}{s_x} + z_x\right), 0, 2^b - 1\right), \end{aligned} \tag{2}$$

where clip(x, y, z) is a function that outputs y if $x < y$, z if $x > z$, and x otherwise. s_x and z_x are a scale factor and a zero point, respectively, which are defined as

$$s_x = \frac{2^b - 1}{\alpha - \beta}, \quad z_x = -\text{round}(\beta s_x) - 2^{b-1}.$$

Dequantization restores the real-valued number x_r from the quantized integer x_q as follows:

$$\begin{aligned} x_r' &= \text{dequantize}(x_q, s_x, z_x) \\ &= (x_q - z_x) \cdot s_x. \end{aligned} \tag{3}$$

The dequantized value x_r' is an approximation of the original value x_r due to the quantization error.

To balance the tradeoff between the quantization error and the computation cost, both scale factor and zero point should be adjusted in a different quantization granularity. The quantization with the finest granularity prepares per-element quantization parameters. The quantization with moderate granularity prepares per-channel quantization parameters. The quantization with the coarsest granularity uses per-tensor quantization parameters.

3.3.2 Post-Training Quantization

In order to mitigate inference performance degradation, the quantization parameters (i.e., scale factors and zero points) must be adjusted such that the quantization error is minimized. PTQ is a method to adjust the quantization parameters of the trained model [24]. This is done by a calibration process [25]. The calibration adjusts the quantization parameters in an offline manner such that the Kullback-Leibler (KL) divergence [26] between the original and quantized distribution of the activations is minimized, given the calibration data (i.e., a small portion of training data) as inputs. Therefore, PTQ is an unsuitable approach to adjust the quantization parameters in an online manner.

3.3.3 Quantization-Aware Training

QAT is a method to adjust the quantization parameters during the training process [13], [14]. To simulate the error associated with the quantization approximation, QAT introduces fake quantization to the activations and the weights. From Eqs. (2) and (3), the fake quantization is defined as

$$x_r' = \text{dequantize}(\text{quantize}(x_r, s_x, z_x), s_x, z_x). \tag{4}$$

Because Eq. (4) is indifferentiable, the backpropagation algorithm cannot be directly applied. To address this issue, the *straight-through estimator* (STE) [27] is used to approximate the gradient of the quantization operation. Thanks to the STE, the quantization parameters can be learnable during the training process [28]. Unlike PTQ, QAT facilitates continuous learning of ML-based IDSs. Considering this point, we adopt QAT as the model quantization.

3.4 Autoencoders

An AE is an unsupervised ML algorithm using NNs [29] and is widely used for anomaly detection and data compression. The AE consists of an encoder and a decoder. The encoder maps the input data to a lower-dimensional latent space and the decoder reconstructs the input data from the latent space. In the training phase, the AE model is trained to minimize the reconstruction error between the input and the output. In the inference phase, the input data is regarded as an anomaly if the reconstruction error e is larger than the predefined threshold θ. Otherwise, it is regarded as benign.

4. Proposed Scheme

4.1 Overview

In this section, we propose a real-time in-kernel IDS based on unsupervised learning. The proposed scheme is designed to detect anomalies in network traffic by calculating the reconstruction error between the input and the output of the AE model trained with QAT. Figure 1 illustrates the flowchart of the proposed scheme. The proposed scheme consists of three parts: data collection (Sect. 4.2), training (Sect. 4.3), and in-kernel anomaly detection (Sect. 4.4). To collect the training data without the need for annotations, we prepare a white list to allow only benign packets to pass through. We assume that the white list contains the IP addresses of trusted clients. The eBPF program first checks whether an incoming packet is in the predefined white list by inspecting its source IP address. If the source IP address is in the white list, the packet is forwarded to the user-space application and

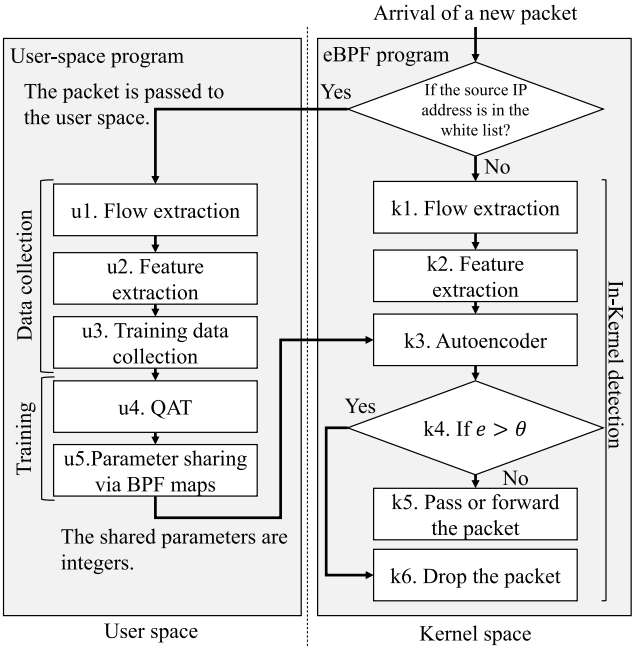


Fig. 1 Flowchart of the proposed scheme.

then the data collection and training phases are executed. Otherwise, the in-kernel anomaly detection is executed. As for outgoing packets, the user-space program inspects the packet header. If the destination IP address is in the white list, the data collection phase is executed.

In the data collection phase, the user-space program extracts the corresponding flow (step u1 in Fig. 1) and calculates the per-packet features (step u2 in Fig. 1). The per-packet features are stored in the database as a feature vector, which is used for the training dataset (step u3 in Fig. 1). In the training phase, the AE model is trained on the collected training data using QAT (step u4 in Fig. 1). After the training phase, the parameters of the trained model are shared with the eBPF program via BPF maps (step u5 in Fig. 1).

In the anomaly detection phase, the eBPF program performs the flow extraction (step k1 in Fig. 1) and the feature extraction (step k2 in Fig. 1). Given the feature vector as an input, the AE model implemented in the eBPF program outputs the reconstructed feature vector. Note that the parameters of the AE model are stored in the BPF maps. The eBPF program calculates the reconstruction error e between the input and the output (step k3 in Fig. 1). If the reconstruction error e is larger than the predefined threshold θ (step k4 in Fig. 1), the packet is regarded as an anomaly and is dropped (step k5 in Fig. 1), otherwise, it is benign and is forwarded to the user space or another device (step k6 in Fig. 1).

4.2 Data Collection

In the data collection phase, the user-space program extracts essential information from the packet header (i.e., source and destination IP addresses ip_s and ip_d, source and destination port numbers $port_s$ and $port_d$, and a protocol number $proto$) and constructs the key $\boldsymbol{k} = \langle ip_s, ip_d, port_s, port_d, proto \rangle$ based on this information. Using this key $\boldsymbol{k}$, it retrieves the corresponding value $\boldsymbol{v_k} = \langle t^{\text{in}}_{\text{last}}, t^{\text{out}}_{\text{last}}, \boldsymbol{ttl}, \boldsymbol{len}^{\text{in}}, \boldsymbol{len}^{\text{out}}, \boldsymbol{iat}, \boldsymbol{idt} \rangle$ from the key-value store. Here, $t^{\text{in}}_{\text{last}}$ (resp. $t^{\text{out}}_{\text{last}}$) represents the arrival (resp. departure) time of the last incoming (resp. outgoing) packet. $\boldsymbol{ttl}$ is a sequence of time-to-live (TTL) values of the packets. $\boldsymbol{len}^{\text{in}}$ and $\boldsymbol{len}^{\text{out}}$ are sequences of packet lengths of incoming and outgoing packets, respectively. Similarly, $\boldsymbol{iat}$ and $\boldsymbol{idt}$ are sequences of packet inter-arrival times (IATs) and inter-departure times (IDTs), respectively. If this retrieval fails, the program adds the key $\boldsymbol{k}$ and the value $\boldsymbol{v_k}$ to the key-value store. Otherwise, it updates the respective value $\boldsymbol{v_k}$ by inspecting the packet header. The detailed update rules are as follows: The program extracts per-packet features (i.e., TTL value ttl, IAT iat, IDT idt, and packet length len) from the packet. It then appends the TTL value ttl extracted from the IP header to the sequence $\boldsymbol{ttl}$. If the packet is an incoming packet, it appends the packet length len to the sequence $\boldsymbol{len}^{\text{in}}$ and the IAT $iat = t - t^{\text{in}}_{\text{last}}$ to the sequence $\boldsymbol{iat}$, respectively, where t denotes the current time. Otherwise, it appends the packet length len to the sequence $\boldsymbol{len}^{\text{out}}$ and the IAT $idt = t - t^{\text{out}}_{\text{last}}$ to the sequence $\boldsymbol{idt}$, respectively.

Table 3 Features used for the AE model.

Symbol	Description
x_1	Protocol number
x_2	Average of TTL
x_3	Average of incoming packet length
x_4	Average absolute deviation of incoming packet length
x_5	Maximum value of incoming packet length
x_6	Minimum value of incoming packet length
x_7	Average of outgoing packet length
x_8	Average absolute deviation of outgoing packet length
x_9	Maximum value of outgoing packet length
x_{10}	Minimum value of outgoing packet length
x_{11}	Average of IAT
x_{12}	Average absolute deviation of IAT
x_{13}	Maximum value of IAT
x_{14}	Minimum value of IAT
x_{15}	Average of IDT
x_{16}	Average absolute deviation of IDT
x_{17}	Maximum value of IDT
x_{18}	Minimum value of IDT

Table 3 presents the features used in this paper. We use a $D = 18$ dimensional feature vector $\boldsymbol{x} = (x_1, \ldots, x_D)$ for the AE model. Note that these features are selected based on the feature importance in the datasets. Here, x_1 can be directly extracted from the packet bytes but x_2–x_{18} are calculated from the sequences $\boldsymbol{ttl}$, $\boldsymbol{len}^{\text{in}}$, $\boldsymbol{len}^{\text{out}}$, $\boldsymbol{iat}$, and $\boldsymbol{idt}$. After the retrieval of the feature vector $\boldsymbol{x}$, it is added to the training data.

Because the feature vector $\boldsymbol{x}$ is not normalized, the min-max normalization converts the feature vector $\boldsymbol{x} = (x_1, \ldots, x_D)$ to the normalized feature vector $\boldsymbol{x}^{\text{norm}} = (x^{\text{norm}}_1, \ldots, x^{\text{norm}}_D)$ using the normalization parameters $\boldsymbol{s}^{\min} = (s^{\min}_1, \ldots, s^{\min}_D)$ and $\boldsymbol{s}^{\max} = (s^{\max}_1, \ldots, s^{\max}_D)$. The min-max normalization is defined as

$$x^{\text{norm}}_i = \frac{x_i - s^{\min}_i}{s^{\max}_i - s^{\min}_i}. \tag{5}$$

4.3 Training in User Space

In this paper, we assume an AE model with $L = 6$ fully connected layers. The former three layers (i.e., $l = 1, 2, 3$) are responsible for an encoder and the latter three layers (i.e., $l = 4, 5, 6$) are responsible for a decoder. Each activation of the lth layer except for the $l = 6$th layer is applied a rectified linear unit (ReLU) function as an activation function. For the lth layer, let the floating-point input, weight, and output be $\mathbf{X}^{(l)} = (x_{i,j}) \in \mathbb{R}^{N \times M^{(l)}}$, $\mathbf{W}^{(l)} = (w_{i,j}) \in \mathbb{R}^{M^{(l)} \times M^{(l+1)}}$, and $\mathbf{Y}^{(l)} = (y_{i,j}) \in \mathbb{R}^{N \times M^{(l+1)}}$, respectively, where N and $M^{(l)}$ denote a batch size and a dimension of the input of the lth layer. Therefore, the lth layer operation is defined as

$$\mathbf{Y}^{(l)} = \sigma^{(l)}(\mathbf{X}^{(l)} \cdot \mathbf{W}^{(l)}), \tag{6}$$

where $\sigma^{(l)}(\cdot)$ denote the activation function of the lth layer. The output $\mathbf{Y}^{(l)}$ is passed to the next layer (i.e., the $l + 1$ ($l \leq L - 1$) layer) as an input (i.e., $\mathbf{X}^{(l+1)} = \mathbf{Y}^{(l)}$).

In QAT, the fake quantization (Eq. (4)) is applied to activations $\mathbf{X}^{(l)}$ ($l \in \{1, \ldots, L + 1\}$) and weights $\mathbf{W}^{(l)}$

($l \in \{1,\ldots,L\}$). In other words, the weights and the quantization parameters are optimized by the backpropagation algorithm [30]. In this paper, we quantize both activations $\mathbf{X}^{(l)}$ ($l \in \{1,\ldots,L+1\}$) and weights $\mathbf{W}^{(l)}$ ($l \in \{1,\ldots,L\}$) with the per-tensor quantization granularity. The respective quantization parameters are defined as $s_x^{(l)}$, $z_x^{(l)}$, $s_w^{(l)}$, and $z_w^{(l)}$. For each lth layer ($l \in \{1,\ldots,L\}$), the optimized weight matrix $\mathbf{W}^{(l)}$ is transformed into the 8-bit quantized weight matrix $\mathbf{W}_q^{(l)}$ according to Eq. (2).

The reconstruction error e_i between the ith input vector $\boldsymbol{x}_i \in \mathbf{X}^{(1)}$ ($\mathbf{X}^{(1)} = (\boldsymbol{x}_1,\ldots,\boldsymbol{x}_N)$) and the ith output vector $\boldsymbol{y}_i \in \mathbf{Y}^{(L)}$ ($\mathbf{Y}^{(L)} = (\boldsymbol{y}_1,\ldots,\boldsymbol{y}_N)$) is defined as residual sum of squares (RSS) as follows:

$$e_i = \sum_{j=1}^{D} \left(x_{i,j}^{(1)} - y_{i,j}^{(L)}\right)^2. \tag{7}$$

Inspired by [31], threshold value θ is defined as $\theta = \mu_{\text{err}} + \sigma_{\text{err}}$, where μ_{err} and σ_{err} are the mean and the standard deviation of the reconstruction error for all samples $\boldsymbol{x}_i$ in the training data, respectively.

Because the eBPF program cannot handle floating-point arithmetic, the parameters θ, $\boldsymbol{s}^{\min}$, $\boldsymbol{s}^{\max}$, $\{s_x^{(l)}\}_{l=1,\ldots,L+1}$, $\{z_x^{(l)}\}_{l=1,\ldots,L+1}$, $\{s_w^{(l)}\}_{l=1,\ldots,L}$, and $\{z_w^{(l)}\}_{l=1,\ldots,L}$ are transformed into 32-bit fixed-point numbers with lower 16-bit decimal part. Furthermore, the integer parameters θ, $\boldsymbol{s}^{\min}$, $\boldsymbol{s}^{\max}$, $\{s_x^{(l)}\}_{l=1,\ldots,L+1}$, $\{z_x^{(l)}\}_{l=1,\ldots,L+1}$, $\{\mathbf{W}_q^{(l)}\}_{l=1,\ldots,L}$, $\{s_w^{(l)}\}_{l=1,\ldots,L}$, and $\{z_w^{(l)}\}_{l=1,\ldots,L}$ are stored in BPF maps. In the in-kernel anomaly detection, the eBPF program calculates the reconstruction error using these parameters stored in the BPF maps.

4.4 In-Kernel Anomaly Detection

When a new packet arrives at the TC ingress, the corresponding eBPF program inspects the packet header. If the source IP address of the incoming packet is not in the white list, the eBPF program, which is attached to the TC ingress, performs in-kernel anomaly detection. The eBPF program extracts the corresponding value $\boldsymbol{v_k}$ from the BPF map using the five-tuple key $\boldsymbol{k}$. If the retrieval is successful, the eBPF program updates the respective value $\boldsymbol{v_k}$ in the same manner as described in Sect. 4.2. If the eBPF program fails to retrieve the flow, it adds the key $\boldsymbol{k}$ and the corresponding value $\boldsymbol{v_k}$ to the BPF map. Similarly, the eBPF program attached to the TC egress inspects outgoing packets and updates the respective value in the BPF map. These operations are performed for a certain period of time per flow.

After this period, the eBPF program performs the in-kernel anomaly detection, which is inspired by [9], [10]. The eBPF program computes a feature vector $\boldsymbol{x} = (x_1,\ldots,x_D)$ by extracting the corresponding value $\boldsymbol{v_k}$ from the BPF map using the key $\boldsymbol{k}$. In what follows, all calculations are performed using fixed-point or integer-only arithmetic.

According to Eq. (5), the feature vector $\boldsymbol{x}$ is normalized into the normalized feature vector $\boldsymbol{x}^{\text{norm}}$. The normalized feature vector $\mathbf{X}^{(1)} = (\boldsymbol{x}^{\text{norm}})$ is quantized into the 8-bit fixed-point matrix $\mathbf{X}_q^{(1)} = (\boldsymbol{x}_q)$ using Eq. (2). From Eqs. (2) and (3), the lth layer operation in the quantized model is defined as:

$$y_{q,n,m}^{(l)} = \sigma^{(l)}\left(z_x^{(l+1)} + S^{(l)} \sum_{k=1}^{M^{(l)}} (x_{q,n,k}^{(l)} - z_x^{(l)})(w_{q,k,m}^{(l)} - z_w^{(l)})\right), \tag{8}$$

where $S^{(l)} = (s_x^{(l)} s_w^{(l)})/s_x^{(l+1)}$, which is represented in a fixed-point number. The activation function $\sigma^{(l)}(x) = \max(0, x)$ of the lth layer is the ReLU function. By applying the quantized layer operation (i.e., Eq. (8)) for all lth layers ($l \in \{1,\ldots,L\}$), the quantized output $\mathbf{Y}_q^{(L)}$ is derived. $\mathbf{Y}_q^{(L)}$ is restored to the fixed-point output $\mathbf{Y}^{(L)}$ using the dequantization operation (i.e., Eq. (3)).

The eBPF program calculates the reconstruction error e between the input $\mathbf{X}^{(1)}$ and the output $\mathbf{Y}^{(L)}$ using Eq. (7). If $e > \theta$, the packet is regarded as anomaly and is dropped in the kernel space. Otherwise, the packet is regarded as benign and is forwarded to the user space or another device.

5. Evaluation

In this section, we evaluate the performance of the proposed scheme in terms of the detection performance (Sect. 5.1) and the packet processing performance (Sect. 5.2).

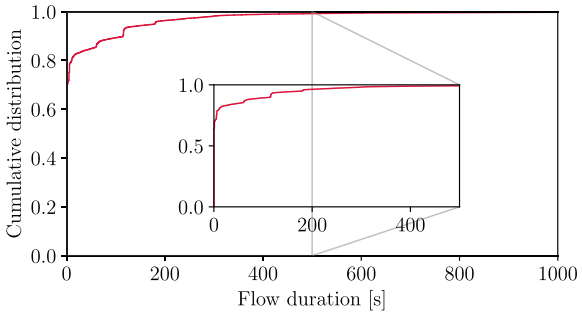
5.1 Detection Performance

5.1.1 Evaluation Scenario

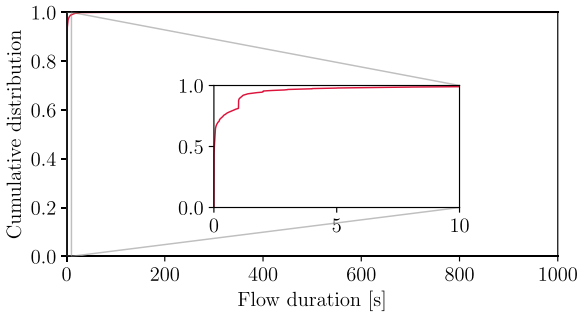
We use two types of datasets: the *CIC-IDS2017* dataset [32] and the *UNSW-NB15* dataset [33] for training and inference. As shown in Table 4, the CIC-IDS2017 (resp. UNSW-NB15) dataset contains 5,313 (resp. 7) source IP addresses and 2,317,916 (resp. 2,065,224) flows. Preprocessing is performed to remove redundant flows and flows with missing values. As a result, the number of flows in CIC-IDS2017 (resp. UNSW-NB15) dataset is reduced to 1,001,463 (resp.

Table 4 Dataset information.

	CIC-IDS2017	UNSW-NB15
Number of source IP addresses	5,313	7
Number of flows	2,317,916	2,065,224
Number of preprocessed flows	1,001,463	1,408,074
Number of source IP addresses stored in the white list	1,042	4
Number of flows used for training	112,004	259,613
Number of flows used for inference	47,418 (# of benign flows is 23,709 and # of abnormal flows is 23,709.)	11,118 (# of benign flows is 5,559 and # of abnormal flows is 5,559.)



(a) CIC-IDS2017 dataset.



(b) UNSW-NB15 dataset.

Fig. 2 Cumulative distribution of flow durations in the training datasets.

1,408,074). 20% of the preprocessed flows are used for training and inference. More specifically, the extracted flows labeled as anomalies are used for inference, along with an equal number of the randomly selected benign-labeled flows. The remaining benign-labeled flows are used for training. Consequently, the number of flows used for training is 112,004 (resp. 259,613), while that used for inference is 47,418 (resp. 11,118) in the CIC-IDS2017 (resp. UNSW-NB15) dataset. It is assumed that the number of source IP addresses stored in the white list is 1,042 (resp. 4) in the CIC-IDS2017 (resp. UNSW-NB15) dataset. Figure 2 illustrates the cumulative distribution of flow durations in the training datasets. We confirm from Fig. 2(a) (resp. Fig. 2(b)) that 99.2% of the flows in the CIC-IDS2017 dataset have a flow duration of 500 seconds or less (resp. 99.0% of the flows in the UNSW-NB15 dataset have a flow duration of 10 seconds or less).

For comparison purpose, we prepare three AE models (i.e., Floating-point model, PTQ-based quantized model, and QAT-based quantized model). Floating-point model contains the model parameters represented in 32-bit floating-point numbers. PTQ-based quantized model is a quantized model of Floating-point model using PTQ. QAT-based quantized model is a quantized model that is trained with QAT. Recall that the proposed scheme adopts QAT-based quantized model. To implement the trained model, we use Pytorch 2.3.0 [34] and the Adam optimizer [35] with a learning rate of 10^{-3} and a batch size of 64.

Our evaluation considers the following metrics based on the confusion matrix: accuracy, precision, recall, and F1-score. The accuracy is defined as

$$\text{accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}.$$

The precision and recall are defined as

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}},$$

respectively. The F1-score is defined as

$$\text{F1-score} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

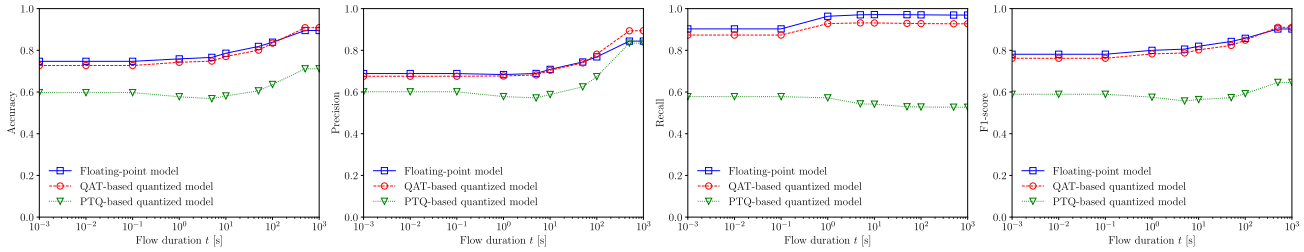
Here, TP indicates the number of true positives (i.e., the number of correctly classified anomaly flows). TN indicates the number of true negatives (i.e., the number of correctly classified benign flows). FP indicates the number of false positives (i.e., the number of incorrectly classified anomaly flows). FN indicates the number of false negatives (i.e., the number of incorrectly classified benign flows).

5.1.2 Detection Accuracy

In this section, we evaluate how the duration t of an individual flow affects the detection performance. The duration t is defined as the elapsed time since the arrival of the first packet belonging to a flow. The larger the value of t, the more accurate the flow statistics become. This suggests that the detection performance tends to improve as the flow duration t increases.

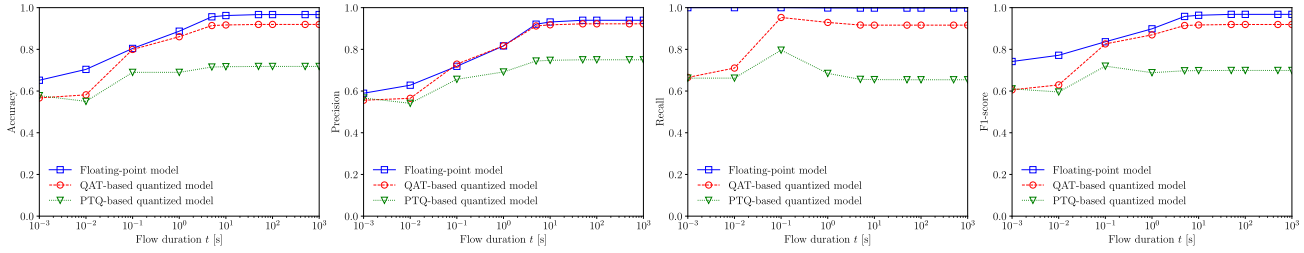
Figure 3 (resp. Fig. 4) illustrates the relationship between the flow duration t and the detection performance of the model trained with the CIC-IDS dataset (resp. the UNSW-NB15 dataset). Comparing Figs. 3 and 4, we observe that the patterns of performance improvement differ between CIC-IDS2017 and UNSW-NB15 datasets. This is due to the fact that the distribution of flow durations in the training dataset is different between the two datasets. Recall that in the CIC-IDS2017 dataset, 99.1% of the flows have a flow duration of 500 seconds or less, while in the UNSW-NB15 dataset, 99.0% of the flows have a flow duration of 10 seconds or less, as shown in Fig. 2. In particular, we observe from Fig. 3 (resp. Fig. 4) that the accuracy and the precision of each model improve when $t < 500$ (resp. $t < 10$) and then saturate when $t \geq 500$ (resp. $t \geq 10$). We observe from Fig. 3 that the recall of the floating-point and QAT-based quantized models also improves when $t < 1$ and saturates when $t \geq 1$ in the CIC-IDS2017 dataset. In UNSW-NB15 dataset, the recall of the floating-point model remains consistently 1.0, while that of the QAT-based quantized model improves when $t < 10^{-1}$ and saturates when $t \geq 10^{-1}$. These results indicate that the proposed IDS can accurately detect malicious flows within 500 seconds and 10 seconds while minimizing missed anomaly detections in case of the CIC-IDS2017 and UNSW-NB15 datasets, respectively.

The achievable detection performance differs among the models. The accuracy, precision, recall, and F1-score of the floating-point model are 0.90, 0.84, 0.97, and 0.90, (resp. 0.97, 0.93, 1.00, and 0.97) in case of the CIC-IDS2017 (resp.



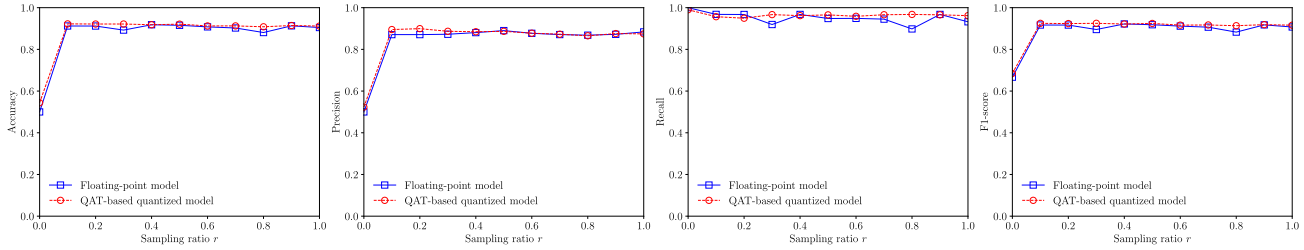
(a) Accuracy. (b) Precision. (c) Recall. (d) F1-score.

Fig. 3 Impact of flow duration t (CIC-IDS2017 dataset).



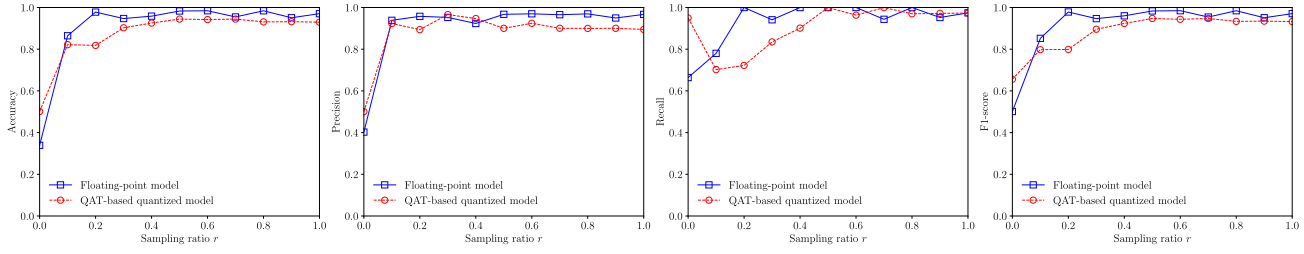
(a) Accuracy. (b) Precision. (c) Recall. (d) F1-score.

Fig. 4 Impact of flow duration t (UNSW-NB15 dataset).



(a) Accuracy. (b) Precision. (c) Recall. (d) F1-score.

Fig. 5 Impact of sampling ratio (Model pretrained with the UNSW-NB15 dataset and retrained with the CIC-IDS2017 dataset).



(a) Accuracy. (b) Precision. (c) Recall. (d) F1-score.

Fig. 6 Impact of sampling ratio (Model pretrained with the CIC-IDS2017 dataset and retrained with the UNSW-NB15 dataset).

UNSW-NB15) dataset. Due to both quantization and fixed-point arithmetic, the PTQ-based quantized model deteriorates the accuracy, precision, recall, and F1-score, compared with the floating-point model. In contrast, the QAT-based quantized model achieves the detection performance comparable to that of the floating-point model. In particular, the accuracy, precision, recall, and F1-score of the QAT-based model reach 0.91, 0.89, 0.93, 0.91 (resp. 0.92, 0.92, 0.92, 0.92) in case of the CIC-IDS2017 (resp. UNSW-NB15) dataset. This is because QAT trains the model while taking the quantization error into account.

5.1.3 Adaptivity to Different Traffic Patterns

In this section, we evaluate how many flows are required to adjust the model parameters to different traffic patterns

through additional training. We define the sampling ratio r as the ratio of the number of flows used for additional training to the number of flows used for training in Table 4. In the evaluation, we use a flow duration of 1,000 seconds, which is the maximum duration. Figure 5 (resp. Fig. 6) depicts how the model, pretrained with the UNSW-NB15 dataset (resp. the CIC-IDS2017 dataset), improves its detection performance when retrained with the CIC-IDS2017 dataset (resp. the UNSW-NB15 dataset) using the sampling ratio r. In Fig. 5 (resp. Fig. 6), the detection performance is measured using the CIC-IDS2017 (resp. the UNSW-NB15) testing dataset. Note that $r = 0$ indicates the pretrained model without the additional training.

We confirm from Figs. 5 and 6 that the detection performance improves as r increases and saturates at $r = 0.1$ except for recall and F1-score. This result suggests that even tuning a model using a small amount of flows can contribute to accurately detecting anomalies. Recall, however, shows different tendencies between datasets. In Fig. 5, the recall consistently reaches 1, regardless of r, which suggests that there are few missed malicious packets, but the accuracy is lacking when $r < 0.1$. In Fig. 6, the floating-point and QAT-based quantized models achieve high recall values when $r \geq 0.2$ and $r \geq 0.5$. In these ranges, the models accurately detect anomalies without missing them. Therefore, the QAT-based model requires 11,200 flows ($r = 0.1$) (resp. 51,923 flows ($r = 0.5$)) to adapt to the traffic patterns of the CIC-IDS2017 (resp. UNSW-NB15) dataset.

5.2 Packet Processing Performance

5.2.1 Evaluation Scenario

For evaluation, we prepare a sender with the 13th Gen Intel Core i7-13700K CPU, memory of 64 GB, a NIC of 10 Gbps, and Ubuntu 22.04 LTS with kernel version 6.2 and a receiver with the Intel Xeon Gold 5317 CPU, memory of 64 GB, a NIC of 10 Gbps, and Ubuntu 22.04 LTS with kernel version 6.8. Both machines are wired via a switching hub. The sender generates UDP packets and sends them to the receiver using the `hping3` command [36]. The receiver runs the eBPF program on the TC of the network interface to perform the in-kernel IDS. To simulate the situations where the low-power devices are used as the sender and the receiver, we restrict the number of CPU cores to one.

For evaluation metrics, we use packet sending rate R_{TX}, packet receiving rate R_{RX}, and CPU utilization. R_{TX} is the number of packets sent per second, which is measured by the sender, while R_{RX} is the number of packets received per second, which is measured by the receiver after the detection process of the IDS. CPU utilization consists of `usr`, `sys`, `iowait`, `soft`, and `idle`, each of which is measured by the receiver using the `mpstat` command [37]. `usr` and `sys` stand for the CPU time spent in the user space and the kernel space, respectively. `iowait` is the CPU time spent waiting for I/O operations. `soft` is the CPU time spent on software interrupts. `idle` is the CPU time spent on idle. Note that eBPF programs consume `soft` time.

For comparison purpose, we prepare three types of packet processing mechanisms: the performance limit, the traditional user-space IDS, and the proposed in-kernel IDS. The traditional user-space IDS is implemented in Python 3.10 and Pytorch 2.3.0 [34] and runs the floating-point AE model in the user space. The proposed in-kernel IDS is implemented in eBPF using the BPF Compiler Collection [38] and runs the quantized AE model in the kernel space. The performance limit is a scenario where the IDS is not executed.

5.2.2 Throughput

Figure 7 depicts how the packet sending rate R_{TX} affects the packet receiving rate R_{RX} among the packet processing mechanisms. This figure indicates that the relationship between R_{TX} and R_{RX} has the following tendencies: (1) R_{RX} is competitive with the line rate in the range of $R_{\mathrm{TX}} = [0, \hat{R}_{\mathrm{TX}}]$. (2) R_{RX} saturates or drops when $R_{\mathrm{TX}} > \hat{R}_{\mathrm{TX}}$. (3) A value of $\hat{R}_{\mathrm{TX}}$ is different among the packet processing mechanisms. The proposed in-kernel IDS shows the line-rate performance when $R_{\mathrm{TX}} \leq 203$ Kpps (i.e., $\hat{R}_{\mathrm{TX}} = 203$ Kpps) and subseqently achieves $R_{\mathrm{RX}} = 224$ Kpps. In contrast, the traditional user-space IDS exhibits the competitive performance with the performance limit when $R_{\mathrm{TX}} \leq 1.0$ Kpps (i.e., $\hat{R}_{\mathrm{TX}} = 1.0$ Kpps) and finally achieves $R_{\mathrm{RX}} = 3.2$ Kpps.

The performance saturation of both schemes comes from CPU resource depletion. Figure 8 illustrates the CPU utilization of the proposed in-kernel IDS and that of the traditional user-space IDS. We confirm that the CPU utilization reaches 100% when $R_{\mathrm{TX}} > \hat{R}_{\mathrm{TX}}$, regardless of the schemes. In other words, the CPU resource is available at $R_{\mathrm{TX}} \leq \hat{R}_{\mathrm{TX}}$. The performance difference between the in-kernel IDS and the user-space one is attributed to the overhead of the context switch between the user space and the kernel space. In Fig. 8(b), the `usr` time in the user-space IDS is dominant because its detection algorithm is implemented in the user space. When $R_{\mathrm{TX}} > \hat{R}_{\mathrm{TX}}$, both `sys` time and `soft` time

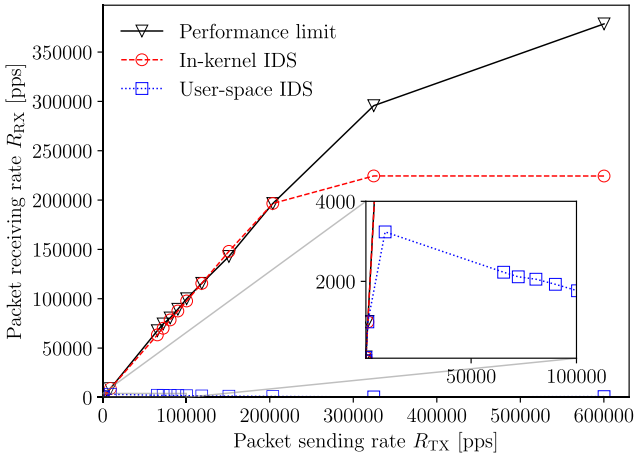
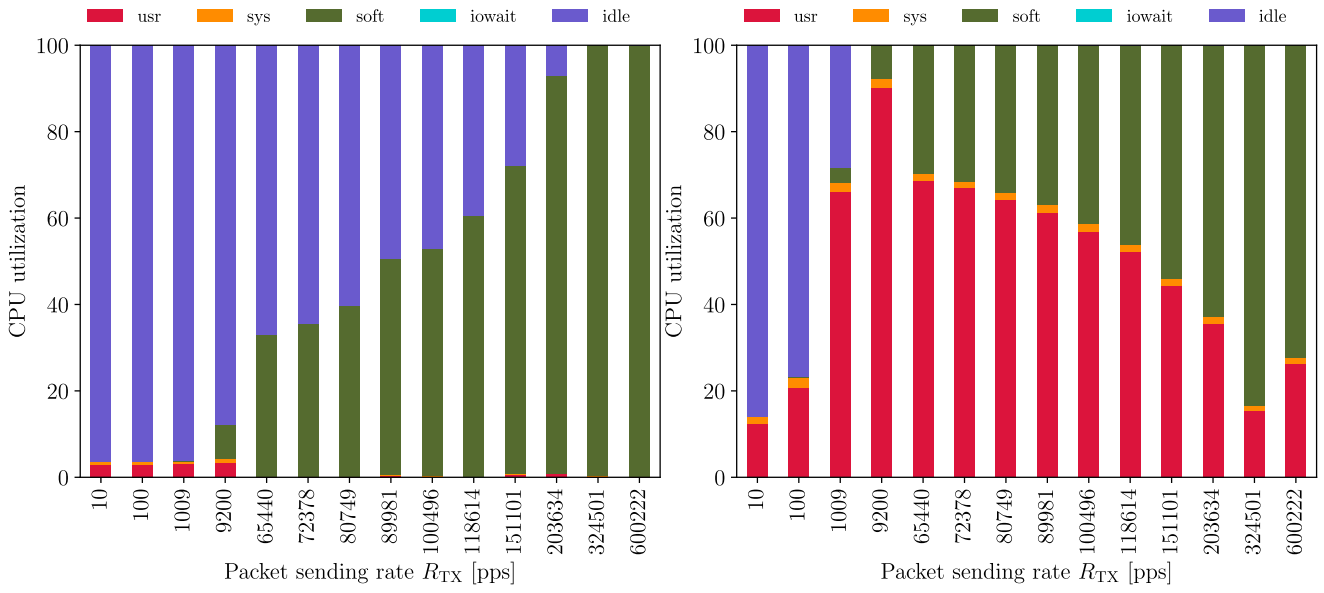


Fig. 7 Impact of R_{TX} on R_{RX} among packet processing mechanisms.



(a) In-kernel IDS.

(b) User-space IDS.

Fig. 8 CPU utilization.

in the user-space IDS increase due to the huge overhead of the context switching cost. In contrast, the in-kernel IDS can eliminate the overhead because the detection algorithm works in the kernel space, as shown in Fig. 8(a). Therefore, its `soft` time becomes dominant.

6. Conclusion

In this paper, we proposed a framework for a real-time in-kernel IDS based on unsupervised learning. The proposed scheme collects benign packets as the training data to be passed through a white list and trains the AE model to take into account the quantization. It deploys the AE model trained with QAT on the eBPF program to perform the in-kernel anomaly detection. Through the evaluation, we demonstrated that the proposed scheme achieves the competitive accuracy with the floating-point model running on the user space. Furthermore, the proposed scheme can fine-tune the pretrained model with the additional training to adapt to the different traffic patterns. From the viewpoint of the packet processing performance, the proposed in-kernel IDS serves up to 224 Kpps in the line rate, which is much higher than the traditional user-space IDS. In future work, we plan to investigate the applicability of the proposed scheme to express data path (XDP) and a kernel-bypassing approach (i.e., AF_XDP).

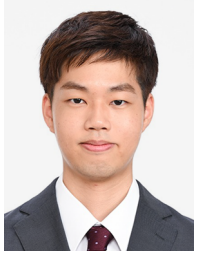
Acknowledgments

This work was supported in part by the Japan Society for the Promotion of Science (JSPS) KAKENHI (B) under Grant 24K02931 and the JSPS Grant-in-Aid for Young Scientists under Grant 23K16869, Japan.

References

[1] H. Taguchi, T. Hara, and S. Kasahara, "On performance analysis of quantized unsupervised learning for in-kernel flow-based intrusion detection systems," Proc. International Conference Emerging Technologies for Communications, p.1, Nov. 2024.
[2] K. Ilgun, R. Kemmerer, and P. Porras, "State transition analysis: A rule-based intrusion detection approach," IEEE Trans. Softw. Eng., vol.21, no.3, pp.181–199, March 1995.
[3] R. Chalapathy and S. Chawla, "Deep learning for anomaly detection: A survey," arXiv preprint, no.arXiv:1901.03407, Jan. 2019.
[4] G. Pang, C. Shen, L. Cao, and A.V.D. Hengel, "Deep learning for anomaly detection: A review," ACM Comput. Surv., vol.54, no.2, pp.38:1–38:38, March 2021.
[5] F. Parola, R. Procopio, R. Querio, and F. Risso, "Comparing user space and in-kernel packet processing for edge data centers," ACM SIGCOMM Computer Communication Review, vol.53, no.1, pp.14–29, Jan. 2023.
[6] M. Bachl, J. Fabini, and T. Zseby, "A flow-based IDS using machine learning in eBPF," arXiv preprint, no.arXiv:2102.09980, March 2022.
[7] N. Anand, S. M A, P.K. Aakula, R.B. Ponnuru, R. Patan, and C.R.P. Reddy, "Enhancing intrusion detection against denial of service and distributed denial of service attacks: Leveraging extended Berkeley packet filter and machine learning algorithms," IET Communications, vol.19, no.1, pp.e12879:1–e12879:18, 2025.
[8] J. Gallego-Madrid, I. Bru-Santa, A. Ruiz-Rodenas, R. Sanchez-Iborra, and A. Skarmeta, "Machine learning-powered traffic processing in commodity hardware with eBPF," Computer Networks, vol.243, pp.110295:1–110295:9, April 2024.
[9] T. Hara and M. Sasabe, "Practicality of in-kernel/user-space packet processing empowered by lightweight neural network and decision tree," Computer Networks, vol.240, pp.110188:1–110188:18, Feb. 2024.
[10] J. Zhang, P. Chen, Z. He, H. Chen, and X. Li, "Real-time intrusion detection and prevention with neural network in kernel using eBPF," Proc. IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), pp.416–428, June 2024.
[11] B. Gregg, BPF Performance Tools: Linux System and Application

Observability, First ed., Pearson Education, Hoboken, 2019.
[12] D. Calavera and L. Fontana, Linux Observability With BPF: Advanced Programming for Performance Analysis and Networking, Oreilly & Associates, Beijing China; Sebastopol, CA, Dec. 2019.
[13] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," Proc. IEEE Conference on Computer Vision and Pattern Recognition, pp.2704–2713, 2018.
[14] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," arXiv preprint, no.arXiv:1806.08342, June 2018.
[15] H.V. Wieren, Signature-based DDoS attack mitigation: Automated generating rules for extended Berkeley packet filter and express data path, Ph.D. thesis, University of Twente, 2019.
[16] Y. Choe, J.S. Shin, S. Lee, and J. Kim, "eBPF/XDP based network traffic visualization and DoS mitigation for intelligent service protection," Proc. Advances in Internet, Data and Web Technologies, L. Barolli, Y. Okada, and F. Amato, eds., Lecture Notes on Data Engineering and Communications Technologies, vol.47, Cham, pp.458–468, Springer International Publishing, 2020.
[17] H.J. Hadi, M. Adnan, Y. Cao, F.B. Hussain, N. Ahmad, M.A. Alshara, and Y. Javed, "iKern: Advanced intrusion detection and prevention at the kernel level using eBPF," Technologies, vol.12, no.8, pp.122:1–122:23, Aug. 2024.
[18] H.M. Demoulin, I. Pedisich, N. Vasilakis, V. Liu, B.T. Loo, and L.T.X. Phan, "Detecting asymmetric application-layer denial-of-service attacks in-flight with FineLame," Proc. USENIX Annual Technical Conference (USENIX ATC 19), pp.693–708, 2019.
[19] T. Hara and M. Sasabe, "On practicality of kernel packet processing empowered by lightweight neural network and decision tree," Proc. International Conference on Network of Future (NoF), pp.89–97, Oct. 2023.
[20] H. Benmaghnia, M. Martel, and Y. Seladji, "Code generation for neural networks based on fixed-point arithmetic," ACM Trans. Embed. Comput. Syst., vol.23, no.5, pp.1–28, Sept. 2022.
[21] A.R. Omondi, Computer Arithmetic Systems: Algorithms, Architecture and Implementation, Prentice Hall International, UK, 1994.
[22] A. Gholami, S. Kim, Z. Dong, Z. Yao, M.W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," arXiv preprint, no.arXiv:2103.13630, June 2021.
[23] M. Nagel, M. Fournarakis, R.A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A white paper on neural network quantization," arXiv preprint, no.arXiv:2106.08295, June 2021.
[24] H. Wu, P. Judd, X. Zhang, M. Isaev, and P. Micikevicius, "Integer quantization for deep learning inference: Principles and empirical evaluation," arXiv preprint, no.arXiv:2004.09602, April 2020.
[25] S. Migacz, "8-bit Inference with TensorRT," GPU Technology Conference, 2017.
[26] S. Kullback and R.A. Leibler, "On information and sufficiency," Ann. Math. Statist., vol.22, no.1, pp.79–86, 1951.
[27] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," arXiv preprint, no.arXiv:1308.3432, Aug. 2013.
[28] S.K. Esser, J.L. McKinstry, D. Bablani, R. Appuswamy, and D.S. Modha, "Learned step size quantization," arXiv preprint, no.arXiv:1902.08153, May 2020.
[29] G.E. Hinton and R.R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," Science, vol.313, no.5786, pp.504–507, July 2006.
[30] D.E. Rumelhart, G.E. Hinton, and R.J. Williams, "Learning representations by back-propagating Errors," Nature, vol.323, no.6088, pp.533–536, Oct. 1986.
[31] M.R. Shahid, G. Blanc, Z. Zhang, and H. Debar, "Anomalous communications detection in IoT networks using sparse autoencoders," Proc. IEEE International Symposium on Network Computing and Applications (NCA), pp.1–5, Sept. 2019.
[32] I. Sharafaldin, A. Habibi Lashkari, and A.A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," Proc. International Conference on Information Systems Security and Privacy, Funchal, Madeira, Portugal, pp.108–116, SCITEPRESS - Science and Technology Publications, 2018.
[33] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," Proc. Military Communications and Information Systems Conference (MilCIS), pp.1–6, Jan. 2015.
[34] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," arXiv preprint, no.arXiv:1912.01703, Dec. 2019.
[35] D.P. Kingma and J. Ba, "Adam: A method for stochastic optimization," arXiv preprint, no.arXiv:1412.6980, Jan. 2017.
[36] antirez, "antirez/hping," https://github.com/antirez/hping, 2024. Accessed: 25 Dec. 2024.
[37] sysstat, "mpstat - Display CPU Usage," https://github.com/sysstat/sysstat, 2024. Accessed: 17 Dec. 2024.
[38] iovisor, "BPF Compiler Collection (BCC)," https://github.com/iovisor/bcc, 2024. Accessed: 17 Dec. 2024.



Hotaka Taguchi received the M.Eng. degrees from Nara Institute of Science and Technology, Nara, Japan, in 2025. He is currently an engineer with NTT DOCOMO, where he works on cloud technologies and network systems. His research interests include AI/ML-powered networking, eBPF/XDP, and real-time intrusion detection systems.



Takanori Hara received the M.Eng. and Ph.D. degrees from Nara Institute of Science and Technology, Nara, Japan, in 2018 and 2021. He is currently an Associate Professor with the Division of Information Science, Graduate School of Science and Technology, Nara Institute of Science and Technology, Japan. His research interests include AI/ML empowered networking, network virtualization, eBPF/XDP, and game-theoretic approaches.



Shoji Kasahara received the B.Eng., M.Eng., and Dr. Eng. degrees from Kyoto University, Kyoto, Japan, in 1989, 1991, and 1996, respectively. Currently, he is a Professor of Division of Information Science, Nara Institute of Science and Technology, Nara, Japan. His research interests include stochastic modeling and analytics of large-scale complex systems based on computer/communication networks.